Boutique App

Local Deployment Runbook — Every PowerShell Command, Start to Finish

Prepared for: Alamgir | Folder: devops-ai-playbook / projects / boutique-microservices | Platform: Windows + PowerShell + Docker Desktop

---

This is a self-contained runbook for deploying the app locally using this exact devops-ai-playbook folder, independently and from scratch, any time in the future — a new computer, a fresh clone, after Docker Desktop was reset, or just starting clean. All six bugs found during the original setup (missing Checkout route, missing environment variable, missing database column, invalid UUID, and the two API response-shape mismatches) are already fixed permanently in the project files, including the database's own initialization script — so none of the one-off fix commands from that troubleshooting session are needed again. Everything below is the clean path.

> **How to use this:** Every command assumes PowerShell is already open with the current folder set correctly for that step. Run commands one at a time and let each finish before pasting the next.

Part 1 — One-Time Prerequisite Check

Skip this part if Git, Docker Desktop, and Node.js are already installed and working on this machine.

1 Confirm Git is installed

```
git --version
```

Expect to see: git version 2.x.x

2 Confirm Docker Desktop is installed and running

```
docker version
```

Expect to see: Both a Client and a Server section print with no errors. If it errors, open the Docker Desktop app and wait for it to say “Docker Desktop is running,” then retry.

3 Confirm Node.js and npm are installed

```
node -v
npm -v
```

Expect to see: Node v20.x.x and an npm version number

Part 2 — Deploy From Scratch

4 Go to the project folder

```
cd "C:\Users\MD
Alamgir\Desktop\devops-ai-playbook\projects\boutique-microservices"
```

5 Make sure Docker Desktop's engine is running

```
docker info
```

Expect to see: A long block of system info with no “cannot connect” error. If it errors, open Docker Desktop, wait for it to finish starting, then retry.

6 Set up environment files (only if .env doesn't already exist)

Each backend service reads its own .env file. If they're already present in the folder, skip this step.

```
Get-ChildItem -Recurse -Filter "*.env.example"
```

Expect to see: A list of .env.example files under backend\services*. Copy each one found to the same name without “.example” before continuing, e.g.: Copy-Item backend\services\auth\.env.example backend\services\auth\.env

7 Install frontend dependencies

Run from: `boutique-microservices`

```
cd frontend
npm install
```

Expect to see: “added N packages” with no red error lines (yellow warnings are fine)

8 Build the frontend static files

Run from: `boutique-microservices\frontend`

The frontend container is plain nginx serving this build folder directly — it is not built by Docker, so this step must run before Docker Compose starts.

```
npm run build
```

Expect to see: “Compiled with warnings” (fine) or “Compiled successfully”, ending with “The build folder is ready to be deployed.”

9 Go back to the project root

Run from: `boutique-microservices\frontend`

```
cd ..
```

10 Start every service

Run from: `boutique-microservices`

```
docker compose up -d --build
```

Expect to see: A list of 10 containers, each ending in “Started”, “Running”, or “Healthy”. This step can take a few minutes the first time.

11 Check every container is healthy

```
docker compose ps
```

Expect to see: All containers show a State of “Up” (postgres shows “Up (healthy)”)

12 Seed the database with demo products (first run only)

Skip this if the database volume already has data from a previous run.

```
Get-Content .\database\quick-seed.sql | docker exec -i boutique-postgres psql -U
postgres -d products_db
```

Expect to see: Mostly INSERT confirmations. A couple of harmless duplicate-key/foreign-key lines may appear — the 5 core products still get seeded correctly.

13 Open the app

Visit in your browser:

```
http://localhost:3000
```

Part 3 — Everyday Commands

Commands you'll reach for again and again while developing — not one-time setup, just the normal day-to-day loop.

14 View logs for one service (swap the service name)

```
docker compose logs --tail=50 orders
```

Expect to see: The service name is one of: gateway, auth, product-service, order-service, orders, user-service, frontend, postgres

15 Rebuild a backend service after editing its code

Needed for gateway, auth, product-service, order-service, orders, and user-service — these are real Docker builds.

```
docker compose build --no-cache orders
docker compose up -d --force-recreate orders
```

16 Apply a frontend code change

Run from: `boutique-microservices`

The frontend has no Docker build step — just rebuild the static files and restart nginx.

```
cd frontend
npm run build
cd ..
docker compose restart frontend
```

17 Stop everything (keeps your data)

```
docker compose down
```

18 Start everything again later

```
docker compose up -d
```

Expect to see: No --build needed unless source files changed since the last build.

19 Full reset — wipe the database and start completely clean

Use this if things get into a broken state you can't otherwise explain. You will need to re-run Step 12 afterward.

```
docker compose down -v
```

Why this stays fixed: Since the checkout route, environment variables, database schema, and API response-shape fixes are all saved directly in this project's files (including database/init/20-init-schema.sql), a full reset in Step 19 rebuilds a database that already includes every fix — you will not hit any of the six original bugs again.